Bikkad IT
Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 11: File Handling & Serialization (OOP-4)

Practice Questions with Python Code Answers
Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Operator Overloading (Magic Methods)
2. Writing to Text Files
3. Append Mode
4. Writing Many Files & File Locations
5. Reading Files: read()
6. Reading Files: readline(), readlines() & Line by Line
7. Context Manager (with)
8. Reading in Chunks, seek() & tell()
9. Binary Files & Other Data Types
10. JSON Serialization: dump() & load()
11. JSON with Tuples, Nested Dicts & Objects
12. Pickling & Pickle vs JSON

72 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad
+91 77220 82226 Chh. Sambhajinagar
+91 77220 82227 Nashik
+91 77220 82224 Hadapsar, Pune

Session 11: File Handling & Serialization (OOP-4) - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Operator Overloading (Magic Methods), Writing to Text Files, Append Mode, Writing Many Files & File Locations, Reading Files: read(), Reading Files: readline(), readlines() & Line by Line, Context Manager (with), Reading in Chunks, seek() & tell(), Binary Files & Other Data Types, JSON Serialization: dump() & load(), JSON with Tuples, Nested Dicts & Objects, Pickling & Pickle vs JSON
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Operator Overloading (Magic Methods)

Q1. Add Two Points

BEGINNER

Create class Point(x, y) with __add__ and __str__. Print Point(2, 5) + Point(3, 1).

Solution:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Point({self.x}, {self.y})"

print(Point(2, 5) + Point(3, 1))
```

Output:

```
Point(5, 6)
```

Q2. Subtract Time Durations

BEGINNER

Create class Duration(minutes) with __sub__ and __str__. Subtract Duration(90) - Duration(25).

Solution:

```
class Duration:
    def __init__(self, minutes):
        self.minutes = minutes

    def __sub__(self, other):
        return Duration(self.minutes - other.minutes)

    def __str__(self):
        return f"{self.minutes} min"

print(Duration(90) - Duration(25))
```

Output:

```
65 min
```

Q3. Compare Products by Price

INTERMEDIATE

Create class Product(name, price) with __lt__ and __eq__. Compare two products and find the cheaper one using min().

Solution:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __lt__(self, other):
        return self.price < other.price

    def __eq__(self, other):
        return self.price == other.price

a = Product("Pen", 20)
b = Product("Marker", 45)
print(a < b)
print(a == b)
print(min(a, b).name)
```

Output:

```
True
False
Pen
```

Q4. Multiply a Recipe

INTERMEDIATE

Create class Recipe(dict of ingredients in grams) with __mul__ to scale for more people. Scale {"rice": 200, "dal": 100} by 3.

Solution:

```
class Recipe:
    def __init__(self, items):
        self.items = items

    def __mul__(self, n):
        return Recipe({k: v * n for k, v in self.items.items()})

    def __str__(self):
        return str(self.items)

print(Recipe({"rice": 200, "dal": 100}) * 3)
```

Output:

```
{'rice': 600, 'dal': 300}
```

Q5. Matrix Addition

ADVANCED

Create class Matrix(rows) with __add__ for element-wise addition and __str__ printing rows on separate lines.

Solution:

```
class Matrix:
    def __init__(self, rows):
        self.rows = rows

    def __add__(self, other):
        result = []
        for r1, r2 in zip(self.rows, other.rows):
            result.append([a + b for a, b in zip(r1, r2)])
        return Matrix(result)

    def __str__(self):
        return "\n".join(str(r) for r in self.rows)

m = Matrix([[1, 2], [3, 4]]) + Matrix([[10, 20], [30, 40]])
print(m)
```

Output:

```
[11, 22]
[33, 44]
```

Q6. Bank Balance with += and len

ADVANCED

Create class Passbook with a list of transactions. Support passbook += amount (__iadd__), len(passbook) (__len__) and balance property.

Solution:

```
class Passbook:
    def __init__(self):
        self.transactions = []

    def __iadd__(self, amount):
        self.transactions.append(amount)
        return self

    def __len__(self):
        return len(self.transactions)

    @property
    def balance(self):
        return sum(self.transactions)

pb = Passbook()
pb += 5000
pb += -1200
pb += 300
print(len(pb))
print(pb.balance)
```

Output:

```
3
4100
```

2. Writing to Text Files

Q7. Write a Single Line

BEGINNER

Write "Welcome to Bikkad IT" into welcome.txt using open() with mode "w". Then read and print it.

Solution:

```
f = open("welcome.txt", "w")
f.write("Welcome to Bikkad IT")
f.close()

f = open("welcome.txt", "r")
print(f.read())
f.close()
```

Output:

```
Welcome to Bikkad IT
```

Q8. Write Multiple Lines

BEGINNER

Write three student names on separate lines in students.txt using \n, then print the file content.

Solution:

```
f = open("students.txt", "w")
f.write("Om\n")
f.write("Pia\n")
f.write("Raj\n")
f.close()

print(open("students.txt").read())
```

Output:

```
Om
Pia
Raj
```

Q9. writelines() from a List

INTERMEDIATE

Write the list of cities ["Pune", "Nashik", "Nagpur"] to cities.txt using writelines() (add the newlines yourself).

Solution:

```
cities = ["Pune", "Nashik", "Nagpur"]

f = open("cities.txt", "w")
f.writelines([c + "\n" for c in cities])
f.close()

print(open("cities.txt").read())
```

Output:

```
Pune
Nashik
Nagpur
```

Q10. Mode w Overwrites

INTERMEDIATE

Write "Version 1" into log.txt, then open it again in "w" mode and write "Version 2". Print the final content.

Solution:

```
with open("log.txt", "w") as f:
    f.write("Version 1")

with open("log.txt", "w") as f:
    f.write("Version 2")

print(open("log.txt").read())
```

Output:

```
Version 2
```

Q11. Save a Bill

ADVANCED

Write a formatted bill for items {"Tea": (2, 15), "Samosa": (3, 20)} as (qty, rate) into bill.txt with a total line. Print the file.

Solution:

```
items = {"Tea": (2, 15), "Samosa": (3, 20)}
total = 0

with open("bill.txt", "w") as f:
    f.write("Item     Qty  Amount\n")
    for name, (qty, rate) in items.items():
        amount = qty * rate
        total += amount
        f.write(f"{name:<9}{qty:<5}{amount}\n")
    f.write(f"Total: {total}\n")

print(open("bill.txt").read())
```

Output:

```
Item     Qty  Amount
Tea      2    30
Samosa   3    60
Total: 90
```

Q12. Exclusive Create Mode x

ADVANCED

Use mode "x" to create report.txt. Run it again inside try-except to show FileExistsError when the file already exists.

Solution:

```
import os

if os.path.exists("report.txt"):
    os.remove("report.txt")

with open("report.txt", "x") as f:
    f.write("First report")
print("Created")

try:
    with open("report.txt", "x") as f:
        f.write("Again")
except FileExistsError as e:
    print("FileExistsError")
```

Output:

```
Created
FileExistsError
```

3. Append Mode

Q13. Add a Line to a Diary

BEGINNER

Create diary.txt with "Day 1: Started Python" and then append "Day 2: Learnt loops" using mode "a".

Solution:

```
with open("diary.txt", "w") as f:
    f.write("Day 1: Started Python\n")

with open("diary.txt", "a") as f:
    f.write("Day 2: Learnt loops\n")

print(open("diary.txt").read())
```

Output:

```
Day 1: Started Python
Day 2: Learnt loops
```

Q14. Append Creates the File

BEGINNER

Delete visitors.txt if it exists, then open it in "a" mode and write a name. Show the file was created.

Solution:

```python
import os

if os.path.exists("visitors.txt"):
    os.remove("visitors.txt")

with open("visitors.txt", "a") as f:
    f.write("Asha\n")

print(os.path.exists("visitors.txt"))
print(open("visitors.txt").read())
```

Output:

```
True
Asha
```

Q15. Attendance Log

INTERMEDIATE

Append attendance entries for 3 students in a loop to attendance.txt and count the lines afterwards.

Solution:

```python
open("attendance.txt", "w").close()

for name in ["Om", "Pia", "Raj"]:
    with open("attendance.txt", "a") as f:
        f.write(name + ",Present\n")

lines = open("attendance.txt").readlines()
print(len(lines))
print(lines)
```

Output:

```
3
['Om,Present\n', 'Pia,Present\n', 'Raj,Present\n']
```

Q16. Timestamp Log

INTERMEDIATE

Append a log line with a fixed timestamp "2026-10-17 10:30" and message "Server started" to app.log.

Solution:

```
timestamp = "2026-10-17 10:30"

with open("app.log", "w") as f:
    f.write("LOG FILE\n")

with open("app.log", "a") as f:
    f.write(f"[{timestamp}] Server started\n")

print(open("app.log").read())
```

Output:

```
LOG FILE
[2026-10-17 10:30] Server started
```

Q17. Logger Function

ADVANCED

Write log(message, level="INFO") that appends "LEVEL: message" to events.log. Log 3 messages and print the file.

Solution:

```
def log(message, level="INFO"):
    with open("events.log", "a") as f:
        f.write(f"{level}: {message}\n")

open("events.log", "w").close()
log("User logged in")
log("Disk almost full", "WARNING")
log("Payment failed", "ERROR")

print(open("events.log").read())
```

Output:

```
INFO: User logged in
WARNING: Disk almost full
ERROR: Payment failed
```

Q18. Append Only New Entries

ADVANCED

subscribers.txt has "a@x.com". Append emails ["b@x.com", "a@x.com", "c@x.com"] but skip ones already present.

Solution:

```
with open("subscribers.txt", "w") as f:
    f.write("a@x.com\n")

new_emails = ["b@x.com", "a@x.com", "c@x.com"]
existing = set(open("subscribers.txt").read().split())

with open("subscribers.txt", "a") as f:
    for e in new_emails:
        if e not in existing:
            f.write(e + "\n")
            existing.add(e)

print(open("subscribers.txt").read())
```

Output:

```
a@x.com
b@x.com
c@x.com
```

4. Writing Many Files & File Locations

Q19. Create Files in a Loop

BEGINNER

Create day1.txt, day2.txt and day3.txt each containing "Notes for day N".

Solution:

```
for n in range(1, 4):
    with open(f"day{n}.txt", "w") as f:
        f.write(f"Notes for day {n}")

print(open("day2.txt").read())
```

Output:

```
Notes for day 2
```

Q20. Save in a Folder

BEGINNER

Create a folder "reports" using os.makedirs (exist_ok=True) and save summary.txt inside it.

Solution:

```
import os

os.makedirs("reports", exist_ok=True)
with open(os.path.join("reports", "summary.txt"), "w") as f:
    f.write("Monthly summary")

print(os.listdir("reports"))
```

Output:

```
['summary.txt']
```

Q21. One File per Student

INTERMEDIATE

For marks {"Om": 78, "Pia": 91}, create a file per student inside a "results" folder like Om.txt with the marks.

Solution:

```
import os

marks = {"Om": 78, "Pia": 91}
os.makedirs("results", exist_ok=True)

for name, m in marks.items():
    with open(os.path.join("results", name + ".txt"), "w") as f:
        f.write(f"{name} scored {m}")

print(sorted(os.listdir("results")))
print(open("results/Pia.txt").read())
```

Output:

```
['Om.txt', 'Pia.txt']
Pia scored 91
```

Q22. Absolute Path

INTERMEDIATE

Print the absolute path of summary.txt inside reports using os.path.abspath() and check that it exists.

Solution:

```
import os

os.makedirs("reports", exist_ok=True)
open("reports/summary.txt", "w").close()

path = os.path.abspath("reports/summary.txt")
print(os.path.isabs(path))
print(os.path.exists(path))
```

Output:

```
True
True
```

Q23. Monthly Files with pathlib

ADVANCED

Use pathlib.Path to create a folder "sales_2026" and write files Jan.csv, Feb.csv, Mar.csv with a header line. List the files.

Solution:

```
from pathlib import Path

folder = Path("sales_2026")
folder.mkdir(exist_ok=True)

for month in ["Jan", "Feb", "Mar"]:
    (folder / f"{month}.csv").write_text("date,amount\n")

print(sorted(p.name for p in folder.iterdir()))
```

Output:

```
['Feb.csv', 'Jan.csv', 'Mar.csv']
```

Q24. File Size and Cleanup

ADVANCED

Write 3 files with different text lengths into a temp folder, print each file size with os.path.getsize, then delete them all.

Solution:

```
import os

os.makedirs("temp_files", exist_ok=True)
texts = {"a.txt": "Hi",
         "b.txt": "Hello Pune",
         "c.txt": "Bikkad IT Institute"}

for name, text in texts.items():
    with open(os.path.join("temp_files", name), "w") as f:
        f.write(text)

for name in sorted(texts):
    print(name, os.path.getsize(os.path.join("temp_files", name)))

for name in texts:
    os.remove(os.path.join("temp_files", name))
print(os.listdir("temp_files"))
```

Output:

```
a.txt 2
b.txt 10
c.txt 19
[]
```

5. Reading Files: read()

Q25. Read Whole File

BEGINNER

Create quote.txt with "Dream Big! Start Small!! Act Now!!!" and read it completely using read().

Solution:

```
with open("quote.txt", "w") as f:
    f.write("Dream Big! Start Small!! Act Now!!!")

with open("quote.txt") as f:
    content = f.read()
print(content)
```

Output:

```
Dream Big! Start Small!! Act Now!!!
```

Q26. Read First N Characters

BEGINNER

Read only the first 9 characters of quote.txt using read(9).

Solution:

```
with open("quote.txt", "w") as f:
    f.write("Dream Big! Start Small!! Act Now!!!")

with open("quote.txt") as f:
    print(f.read(9))
```

Output:

```
Dream Big
```

Q27. Count Words and Characters

INTERMEDIATE

Write a paragraph to para.txt. Read it and print the number of characters, words and lines.

Solution:

```
text = "Python is easy.\nPython is powerful.\nLearn Python daily."
with open("para.txt", "w") as f:
    f.write(text)

with open("para.txt") as f:
    data = f.read()

print(len(data))
print(len(data.split()))
print(len(data.splitlines()))
```

Output:

```
55
9
3
```

Q28. File Not Found

INTERMEDIATE

Try to read missing.txt inside try-except and print a friendly message.

Solution:

```
try:
    with open("missing.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found. Please check the name.")
```

Output:

```
File not found. Please check the name.
```

Q29. Most Frequent Word

ADVANCED

Read para.txt and find the most frequent word (ignore case and full stops).

Solution:

```
text = "Python is easy.\nPython is powerful.\nLearn Python daily."
with open("para.txt", "w") as f:
    f.write(text)

with open("para.txt") as f:
    words = f.read().lower().replace(".", "").split()

freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1

top = max(freq, key=freq.get)
print(top, freq[top])
```

Output:

```
python 3
```

Q30. Read Twice Problem

ADVANCED

Call read() two times on the same open file. The second call returns an empty string because the cursor is at the end. Print len of both.

Solution:

```
with open("quote.txt", "w") as f:
    f.write("Dream Big!")

with open("quote.txt") as f:
    first = f.read()
    second = f.read()

print(len(first))
print(len(second))
```

Output:

```
10
0
```

6. Reading Files: readline(), readlines() & Line by Line

Q31. readlines() into a List

BEGINNER

Write 3 product names to products.txt and read them into a list using readlines(). Remove the \n using strip().

Solution:

```
with open("products.txt", "w") as f:
    f.write("Pen\nBook\nBag\n")

with open("products.txt") as f:
    lines = f.readlines()

print(lines)
print([l.strip() for l in lines])
```

Output:

```
['Pen\n', 'Book\n', 'Bag\n']
['Pen', 'Book', 'Bag']
```

Q32. readline() One at a Time

BEGINNER

Read the first two lines of products.txt using readline() twice.

Solution:

```
with open("products.txt", "w") as f:
    f.write("Pen\nBook\nBag\n")

with open("products.txt") as f:
    print(f.readline().strip())
    print(f.readline().strip())
```

Output:

```
Pen
Book
```

Q33. Loop Over the File Object

INTERMEDIATE

Read marks.csv (name,marks) line by line using a for loop and print the students who scored above 75.

Solution:

```
with open("marks.csv", "w") as f:
    f.write("Om,72\nPia,91\nRaj,84\nSam,60\n")

with open("marks.csv") as f:
    for line in f:
        name, mark = line.strip().split(",")
        if int(mark) > 75:
            print(name, mark)
```

Output:

```
Pia 91
Raj 84
```

Q34. Line Numbers

INTERMEDIATE

Print each line of products.txt with its line number using enumerate().

Solution:

```
with open("products.txt", "w") as f:
    f.write("Pen\nBook\nBag\n")

with open("products.txt") as f:
    for n, line in enumerate(f, start=1):
        print(n, line.strip())
```

Output:

```
1 Pen
2 Book
3 Bag
```

Q35. Big Log File, Count Errors

ADVANCED

Create a log file with 10,000 lines where every 100th line has "ERROR". Read it line by line (memory-friendly) and count the errors.

Solution:

```
with open("big.log", "w") as f:
    for i in range(1, 10001):
        level = "ERROR" if i % 100 == 0 else "INFO"
        f.write(f"{level} line {i}\n")

errors = 0
with open("big.log") as f:
    for line in f:
        if line.startswith("ERROR"):
            errors += 1

print(errors)
```

Output:

```
100
```

Q36. Sales Total from CSV

ADVANCED

sales.csv has a header "date,amount" and 4 rows. Skip the header with next() and total the amounts.

Solution:

```
with open("sales.csv", "w") as f:
    f.write("date,amount\n")
    f.write("01-10,1200\n02-10,850\n")
    f.write("03-10,1540\n04-10,990\n")

total = 0
with open("sales.csv") as f:
    next(f)
    for line in f:
        date, amount = line.strip().split(",")
        total += int(amount)

print(total)
```

Output:

```
4580
```

7. Context Manager (with)

Q37. Write with with

BEGINNER

Use with open(...) to write "Hello" to hello.txt. Print f.closed after the block.

Solution:

```
with open("hello.txt", "w") as f:
    f.write("Hello")

print(f.closed)
```

Output:

```
True
```

Q38. Read with with

BEGINNER

Read hello.txt using with and print the content.

Solution:

```
with open("hello.txt", "w") as f:
    f.write("Hello")

with open("hello.txt") as f:
    print(f.read())
```

Output:

```
Hello
```

Q39. File Closed Even After Error

INTERMEDIATE

Inside a with block raise a ValueError on purpose. Catch it outside and show the file is still closed.

Solution:

```
try:
    with open("temp.txt", "w") as f:
        f.write("data")
        raise ValueError("Something went wrong")
except ValueError as e:
    print(e)

print(f.closed)
```

Output:

```
Something went wrong
True
```

Q40. Two Files in One with

INTERMEDIATE

Copy the content of source.txt to copy.txt using one with statement that opens both files.

Solution:

```
with open("source.txt", "w") as f:
    f.write("Line A\nLine B\n")

with open("source.txt") as src, open("copy.txt", "w") as dst:
    for line in src:
        dst.write(line)

print(open("copy.txt").read())
```

Output:

```
Line A
Line B
```

Q41. Custom Context Manager Class

ADVANCED

Create class Timer with __enter__ and __exit__ that prints "Start" and "End". Use it with a with block.

Solution:

```
class Timer:
    def __enter__(self):
        print("Start")
        return self

    def __exit__(self, exc_type, exc, tb):
        print("End")
        return False

with Timer():
    total = sum(range(100))
    print(total)
```

Output:

```
Start
4950
End
```

Q42. contextmanager Decorator

ADVANCED

Use contextlib.contextmanager to write open_log(name) that opens a file, yields it, and prints "Log saved" when done.

Solution:

```
from contextlib import contextmanager

@contextmanager
def open_log(name):
    f = open(name, "a")
    try:
        yield f
    finally:
        f.close()
        print("Log saved")

with open_log("audit.log") as log:
    log.write("Report generated\n")
```

Output:

```
Log saved
```

8. Reading in Chunks, seek() & tell()

Q43. tell() the Position

BEGINNER

Write "ABCDEFGHIJ" to letters.txt. Read 4 characters and print the cursor position with tell().

Solution:

```
with open("letters.txt", "w") as f:
    f.write("ABCDEFGHIJ")

with open("letters.txt") as f:
    print(f.read(4))
    print(f.tell())
```

Output:

```
ABCD
4
```

Q44. seek() to Start

BEGINNER

Read the whole file, then seek(0) and read it again.

Solution:

```
with open("letters.txt", "w") as f:
    f.write("ABCDEFGHIJ")

with open("letters.txt") as f:
    print(f.read())
    f.seek(0)
    print(f.read())
```

Output:

```
ABCDEFGHIJ
ABCDEFGHIJ
```

Q45. Jump to a Position

INTERMEDIATE

Use seek(5) to start reading from the 6th character and read 3 characters.

Solution:

```
with open("letters.txt", "w") as f:
    f.write("ABCDEFGHIJ")

with open("letters.txt") as f:
    f.seek(5)
    print(f.read(3))
    print(f.tell())
```

Output:

```
FGH
8
```

Q46. Read in Chunks

INTERMEDIATE

Read letters.txt in chunks of 3 characters using a while loop until read() returns an empty string.

Solution:

```
with open("letters.txt", "w") as f:
    f.write("ABCDEFGHIJ")

with open("letters.txt") as f:
    while True:
        chunk = f.read(3)
        if not chunk:
            break
        print(chunk)
```

Output:

```
ABC
DEF
GHI
J
```

Q47. Copy a Big File in Chunks

ADVANCED

Create a 50,000-character file and copy it to another file in chunks of 4096 characters. Count the chunks and verify the sizes match.

Solution:

```
import os

with open("big.txt", "w") as f:
    f.write("x" * 50000)

chunks = 0
with open("big.txt") as src, open("big_copy.txt", "w") as dst:
    while True:
        data = src.read(4096)
        if not data:
            break
        dst.write(data)
        chunks += 1

print(chunks)
size1 = os.path.getsize("big.txt")
size2 = os.path.getsize("big_copy.txt")
print(size1 == size2)
```

Output:

```
13
True
```

Q48. Overwrite Part of a File

ADVANCED

Open letters.txt in "r+" mode, seek to position 2 and write "xyz". Print the result.

Solution:

```
with open("letters.txt", "w") as f:
    f.write("ABCDEFGHIJ")

with open("letters.txt", "r+") as f:
    f.seek(2)
    f.write("xyz")

print(open("letters.txt").read())
```

Output:

```
ABxyzFGHIJ
```

9. Binary Files & Other Data Types

Q49. Write Numbers as Text

BEGINNER

write() only accepts strings. Try writing 500 directly inside try-except, then write str(500).

Solution:

```
with open("amount.txt", "w") as f:
    try:
        f.write(500)
    except TypeError as e:
        print(e)
    f.write(str(500))

print(open("amount.txt").read())
```

Output:

```
write() argument must be str, not int
500
```

Q50. Write and Read Bytes

BEGINNER

Write the bytes b"Hello" to data.bin using mode "wb" and read them back with "rb".

Solution:

```
with open("data.bin", "wb") as f:
    f.write(b"Hello")

with open("data.bin", "rb") as f:
    data = f.read()

print(data)
print(type(data))
```

Output:

```
b'Hello'
<class 'bytes'>
```

Q51. Store a List in a Text File

INTERMEDIATE

Save the list [10, 20, 30] to nums.txt as text and read it back as a list of integers.

Solution:

```
numbers = [10, 20, 30]

with open("nums.txt", "w") as f:
    f.write(",".join(str(n) for n in numbers))

with open("nums.txt") as f:
    loaded = [int(x) for x in f.read().split(",")]

print(loaded)
print(sum(loaded))
```

Output:

```
[10, 20, 30]
60
```

Q52. Text vs Binary Size

INTERMEDIATE

Write "Namaste" in text mode and "Namaste".encode() in binary mode. Compare file sizes and decode the binary content.

Solution:

```
import os

with open("t.txt", "w", encoding="utf-8") as f:
    f.write("Namaste")

with open("b.bin", "wb") as f:
    f.write("Namaste".encode("utf-8"))

print(os.path.getsize("t.txt"), os.path.getsize("b.bin"))
print(open("b.bin", "rb").read().decode("utf-8"))
```

Output:

```
7 7
Namaste
```

Q53. Copy an Image-Like Binary File

ADVANCED

Create a fake binary file with bytes(range(256)) and copy it byte-for-byte to another file. Verify both are equal.

Solution:

```
with open("original.bin", "wb") as f:
    f.write(bytes(range(256)))

with open("original.bin", "rb") as src:
    with open("copy.bin", "wb") as dst:
        dst.write(src.read())

a = open("original.bin", "rb").read()
b = open("copy.bin", "rb").read()
print(len(a))
print(a == b)
```

Output:

```
256
True
```

Q54. Why Text Files Are Not Enough

ADVANCED

Save the dictionary {"name": "Om", "marks": 85} as text using str(). Read it back and show it is just a string (type str), not a dict. (This is why we need serialization.)

Solution:

```
student = {"name": "Om", "marks": 85}

with open("student.txt", "w") as f:
    f.write(str(student))

with open("student.txt") as f:
    loaded = f.read()

print(loaded)
print(type(loaded))
```

Output:

```
{'name': 'Om', 'marks': 85}
<class 'str'>
```

10. JSON Serialization: dump() & load()

Q55. Save a Dictionary as JSON

BEGINNER

Save {"name": "Om", "marks": 85} to student.json using json.dump() and print the file content.

Solution:

```
import json

student = {"name": "Om", "marks": 85}

with open("student.json", "w") as f:
    json.dump(student, f)

print(open("student.json").read())
```

Output:

```
{"name": "Om", "marks": 85}
```

Q56. Load JSON Back

BEGINNER

Read student.json using json.load() and show that it is a dictionary again.

Solution:

```
import json

with open("student.json", "w") as f:
    json.dump({"name": "Om", "marks": 85}, f)

with open("student.json") as f:
    data = json.load(f)

print(data)
print(type(data))
print(data["marks"])
```

Output:

```
{'name': 'Om', 'marks': 85}
<class 'dict'>
85
```

Q57. Pretty JSON

INTERMEDIATE

Save a product list to products.json with indent=2 so it is easy to read.

Solution:

```
import json

products = [{"name": "Pen", "price": 10},
            {"name": "Bag", "price": 450}]

with open("products.json", "w") as f:
    json.dump(products, f, indent=2)

print(open("products.json").read())
```

Output:

```
[
  {
    "name": "Pen",
    "price": 10
  },
  {
    "name": "Bag",
    "price": 450
  }
]
```

Q58. dumps() and loads()

INTERMEDIATE

Convert a settings dictionary to a JSON string using dumps() and back using loads(), without any file.

Solution:

```
import json

settings = {"theme": "dark", "notifications": True, "font": 14}

text = json.dumps(settings)
print(text)
print(type(text))

back = json.loads(text)
print(back["theme"])
```

Output:

```
{"theme": "dark", "notifications": true, "font": 14}
<class 'str'>
dark
```

Q59. Update a JSON File

ADVANCED

users.json stores a list of users. Load it, add a new user {"name": "Raj", "city": "Goa"}, and save it back.

Solution:

```
import json

with open("users.json", "w") as f:
    json.dump([{"name": "Om", "city": "Pune"}], f)

with open("users.json") as f:
    users = json.load(f)

users.append({"name": "Raj", "city": "Goa"})

with open("users.json", "w") as f:
    json.dump(users, f, indent=2)

print(len(json.load(open("users.json"))))
```

Output:

```
2
```

Q60. Invalid JSON

ADVANCED

Try to load the text "{name: 'Om'}" with json.loads() inside try-except and catch json.JSONDecodeError.

Solution:

```
import json

bad_text = "{name: 'Om'}"

try:
    json.loads(bad_text)
except json.JSONDecodeError as e:
    print("Invalid JSON:", e.msg)
```

Output:

```
Invalid JSON: Expecting property name enclosed in double quotes
```

11. JSON with Tuples, Nested Dicts & Objects

Q61. Tuple Becomes List

BEGINNER

Save the tuple (18.52, 73.85) in JSON and load it back. Show that it comes back as a list.

Solution:

```
import json

location = (18.52, 73.85)
text = json.dumps(location)
back = json.loads(text)

print(text)
print(type(back))
print(tuple(back))
```

Output:

```
[18.52, 73.85]
<class 'list'>
(18.52, 73.85)
```

Q62. Nested Dictionary

BEGINNER

Save a nested dictionary of a student with address {"city": "Pune", "pin": 411018} to JSON and read the city back.

Solution:

```
import json

student = {"name": "Om",
           "address": {"city": "Pune", "pin": 411018}}

with open("nested.json", "w") as f:
    json.dump(student, f)

with open("nested.json") as f:
    data = json.load(f)

print(data["address"]["city"])
```

Output:

```
Pune
```

Q63. Objects are Not JSON Serializable

INTERMEDIATE

Try json.dumps() on a Student object inside try-except and print the TypeError.

Solution:

```
import json

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

try:
    json.dumps(Student("Om", 85))
except TypeError as e:
    print(e)
```

Output:

```
Object of type Student is not JSON serializable
```

Q64. Serialize an Object with __dict__

INTERMEDIATE

Convert a Student object to JSON using its __dict__.

Solution:

```
import json

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Pia", 92)
print(json.dumps(s.__dict__))
```

Output:

```
{"name": "Pia", "marks": 92}
```

Q65. Custom default Function

ADVANCED

Use json.dump(..., default=func) to save a list of Employee objects where func returns obj.__dict__. Then rebuild Employee objects from the JSON.

Solution:

```
import json

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

staff = [Employee("Om", 50000), Employee("Pia", 62000)]

with open("staff.json", "w") as f:
    json.dump(staff, f, default=lambda o: o.__dict__)

with open("staff.json") as f:
    rows = json.load(f)

loaded = [Employee(**r) for r in rows]
print(loaded[1].name, loaded[1].salary)
```

Output:

```
Pia 62000
```

Q66. Nested Objects to JSON

ADVANCED

A Customer has an Address object. Write a to_dict() method in both classes so the whole thing can be saved as JSON.

Solution:

```
import json

class Address:
    def __init__(self, city, pin):
        self.city = city
        self.pin = pin

    def to_dict(self):
        return {"city": self.city, "pin": self.pin}

class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address

    def to_dict(self):
        return {"name": self.name,
                "address": self.address.to_dict()}

c = Customer("Riya", Address("Nashik", 422001))
print(json.dumps(c.to_dict(), indent=2))
```

Output:

```
{
  "name": "Riya",
  "address": {
    "city": "Nashik",
    "pin": 422001
  }
}
```

12. Pickling & Pickle vs JSON

Q67. Pickle a List

BEGINNER

Save the list ["Pen", "Book", 450] to items.pkl using pickle.dump() and load it back.

Solution:

```
import pickle

items = ["Pen", "Book", 450]

with open("items.pkl", "wb") as f:
    pickle.dump(items, f)

with open("items.pkl", "rb") as f:
    loaded = pickle.load(f)

print(loaded)
```

Output:

```
['Pen', 'Book', 450]
```

Q68. Pickle a Tuple Stays a Tuple

BEGINNER

Pickle the tuple (18.52, 73.85) and show that it comes back as a tuple (unlike JSON).

Solution:

```
import pickle

location = (18.52, 73.85)
data = pickle.dumps(location)
back = pickle.loads(data)

print(back)
print(type(back))
```

Output:

```
(18.52, 73.85)
<class 'tuple'>
```

Q69. Pickle a Custom Object

INTERMEDIATE

Pickle a Student object to student.pkl and load it back. Call its method after loading.

Solution:

```
import pickle

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def result(self):
        return "Pass" if self.marks >= 35 else "Fail"

with open("student.pkl", "wb") as f:
    pickle.dump(Student("Om", 72), f)

with open("student.pkl", "rb") as f:
    s = pickle.load(f)

print(s.name, s.result())
```

Output:

```
Om Pass
```

Q70. Pickle Output is Binary

INTERMEDIATE

Show that pickle.dumps() returns bytes while json.dumps() returns a readable string for the same dictionary.

Solution:

```
import pickle
import json

data = {"city": "Pune"}
print(type(pickle.dumps(data)))
print(type(json.dumps(data)))
print(json.dumps(data))
```

Output:

```
<class 'bytes'>
<class 'str'>
{"city": "Pune"}
```

Q71. Pickle vs JSON Comparison

ADVANCED

Save the same set {1, 2, 3} with pickle (works) and with json inside try-except (fails). Print a short conclusion.

Solution:

```
import pickle
import json

data = {1, 2, 3}

back = pickle.loads(pickle.dumps(data))
print(back)

try:
    json.dumps(data)
except TypeError as e:
    print("JSON error:", e)
```

Output:

```
{1, 2, 3}
JSON error: Object of type set is not JSON serializable
```

Q72. Save Game Progress

ADVANCED

Create class Game(player, level, inventory list). Save progress with pickle, change the object, then load the saved copy to "restore" the game.

Solution:

```
import pickle

class Game:
    def __init__(self, player, level, inventory):
        self.player = player
        self.level = level
        self.inventory = inventory

g = Game("Om", 5, ["sword", "shield"])
with open("save.pkl", "wb") as f:
    pickle.dump(g, f)

g.level = 1
g.inventory.clear()

with open("save.pkl", "rb") as f:
    restored = pickle.load(f)

print(g.level, g.inventory)
print(restored.level, restored.inventory)
```

Output:

```
1 []
5 ['sword', 'shield']
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026